The Global Digest

VOL. 1, NO. 007

2026-03-30

FEATURES

Virtual music box

Oona Räisänen · Oona Raisanen (windytan)

A little music project I was writing required a melody be played on a music box. However, the paper-programmable music box I had (pictured) could only play notes on the C major scale. I couldn't easily find a realistic-sounding synthesizer version either. They all seemed to be missing something. Maybe they were too perfectly tuned? I wasn't sure.

Perhaps, if I digitized the sound myself, I could build a flexible virtual instrument to generate just the perfect sample for the piece!

I haven't really made a sampled instrument before, short of perhaps using Impulse Tracker clones with terrible single-sample ones. So I proceeded in an improvised manner. Below I'll post some interesting findings and sound samples of how the instrument developed along the way. There won't be any source code as for now.

By the way, there is a great explanatory video by engineer-guy about the workings of music boxes that will explain some terminology ("pins" and "teeth") used in this post.

Recording samples

The first step was, obviously, to record the sound to be used as samples. I damped my room using towels and mattresses to minimize room echo; this could be added later if desired, but for now it would only make it harder to cleanly splice the audio. The microphone used was the Audio Technica AT2020, and I digitized it using the Behringer Xenyx 302 USB mixer.

I perforated a paper roll to play all the possible notes in succession, and rolled the paper through. The sound of the paper going through the mechanism posed a problem at first, but I soon learned to stop the paper at just the right moment to make way for the sound of the tooth.

Now I had pretty decent recordings of the whole two-octave range. I used Audacity to extract the notes from the recording, and named the files according to the actual playing MIDI pitch. (The music box actually plays a G# major scale, contrary to what's marked on the blank paper rolls.)

The missing notes

Next, we'll need to generate the missing notes that don't belong in the scale of this music box. Because pitch is proportional to the speed of vibration, this could be done by simply speeding up or slowing down an adjacent note by just the right factor. In equal temperament tuning, this factor would be the 12th root of 2, or roughly 1.05946. Such scaling is straightforward to do on the command line using SoX, for instance (sox c1.wav c_sharp1.wav speed 1.05946).

This method can also be used to generate whole new octaves; for example, a transposition of +8 semitones would have a ratio of (12 √2) 8 ≈ 1.5874. Inter-note variance could be retained by using a random source file for each resampled note. But large-interval transpositions would probably not sound very good due to coloring in the harmonic series.

Here's a table of some intervals and the corresponding speed ratios in equal temperament:

–3 = (12 √2) –3 ≈ 0.840896 –2 = (12 √2) –2 ≈ 0.890899 –1 = (12 √2) –1 ≈ 0.943874 +1 = (12 √2) 1 ≈ 1.059463 +2 = (12 √2) 2 ≈ 1.122462 +3 = (12 √2) 3 ≈ 1.189207

First test!

Now I could finally write a script to play my melody!

(HTML5 audio: Music box notes.)

It sounds pretty good already - there's no obvious noise and the samples line up seamlessly even though they were just naively glued together sample by sample. There's a lot of power in the lower harmonics, probably because of the big cardboard box I used, but this can easily be changed by EQ if we want to give the impression of a cute little music box.

Adding errors

The above sound still sounded quite artificial, I think mostly because simultaneous notes start on the same exact millisecond. There seems to be a small timing variance in music boxes that is an important contributor to their overall delicate sound. In the below sample I added a timing error from a normal distribution with a standard deviation of 11 milliseconds. It sounds a lot better already!

(HTML5 audio: Music box notes.)

Other sounds from the teeth

If you listen to recordings of music boxes you can occasionally hear a high-pitched screech as well. It sounds a bit like stopping a tuning fork or guitar string with a metal object. That's why I thought it must be the sound of the pin stopping a vibrating tooth just before playing another note on the same tooth.

Sure enough, this sound could always be heard by playing the same note twice in quick succession. I recorded this sound for each tooth and added it to my sound generator. The sound will be generated only if the previous note sample is still playing, and its volume will be scaled in proportion to the tooth's envelope amplitude at that moment. Also, it will silence the note. The amount of silence between the screech and the next note will depend on a tempo setting.

Adding this resonance definitely brings about a more organic feel:

(HTML5 audio: Music box notes.)

The wind-up mechanism

Potentially Coming to a Browser :near() You

Daniel Schwarz · CSS-Tricks

Just before we wrapped up 2025, I saw this proposal for :near() , a pseudo-class that would match if the pointer were to go near the element. By how much? Well, that would depend on the value of the argument provided. Thomas Walichiewicz , who proposed :near() , suggests that it works like this:

button:near(3rem) { /* Pointer is within 3rem of the button */ }

For those wondering, yes, we can use the Pythagorean theorem to measure the straight-line distance between two elements using JavaScript (“Euclidean distance” is the mathematical term), so I imagine that that’s what would be used behind the scenes here. I have some use cases to share with you, but the demos will only be simulating :near() since it’s obviously not supported in any web browser. Shall we dig in?

Visual effects

Without question, :near() could be used for a near-infinite (sorry) number of visual effects:

div { /* Div is wow */

&:near(3rem) { /* Div be wowzer */ }

&:near(1rem) { /* Div be woahhhh */ } }

Dim elements until :near()

To reduce visual clutter, you might want to dim certain components until users are near them. :near() could be more effective than :hover in this scenario because users could have trouble interacting with the components if they have limited visibility, and so being able to trigger them “earlier” could compensate for that to some degree. However, we have to ensure accessible color contrast, so I’m not sure how useful :near() can be in this situation.

button:not(:near(3rem)) { opacity: 70%; /* Or...something */ }

Hide elements until :near()

In addition to dimming components, we could also hide components (as long as they’re not important, that is). This, I think, is a better use case for :near() , as we wouldn’t have to worry about color contrast, although it does come with a different accessibility challenge.

So, you know when you hover over an image and a share button appears? Makes sense, right? Because we don’t want the image to be obscured, so it’s hidden initially. It’s not optimal in terms of UX, but it’s nonetheless a pattern that people are familiar with, like on Pinterest for example.

And here’s how :near() can enhance it. People know or suspect that the button’s there, right? Probably in the bottom-right corner? They know roughly where to click, but don’t know exactly where, as they don’t know the size or offset of the button. Well, showing the button when :near() means that they don’t have to hover so accurately to make the button appear. This scenario is pretty similar to the one above, perhaps with different reasons for the reduced visibility.

However, we need this button to be accessible (hoverable, focusable, and find-in-pageable). For that to happen, we can’t use:

display: hidden (not hoverable, focusable, or find-in-page-able)

visibility: hidden (also not hoverable, focusable, or find-in-page-able)

opacity: 0 (there’s no way to show it once it’s been found by find-in-page)

That leaves us with content-visibility: hidden , but the problem with hiding content using content-visibility: hidden (or elements with display: none) is that they literally disappear, and you can’t be near what simply isn’t there. This means that we need to reserve space for it, even if we don’t know how much space.

Now, :near() isn’t supported in any web browser, so in the demo below, I’ve wrapped the button in a container with 3rem of padding , and while that container is being :hover ed, the button is shown. This increases the size of the hoverable region (which I’ve made red, so that you can see it) instead of the actual button. It essentially simulates button:near(3rem) .

CodePen Embed Fallback

But how do we hide something while reserving the space?

First, we declare contain-intrinsic-size: auto none on the hidden target. This ensures that it remains a specific size even as something changes (in this case, even as its content is hidden). You can specify a for either value, but in this case auto means whatever the rendered size was. none , which is a required fallback value, can also be a , but we don’t need that at all, hence “ none .”

The problem is, the rendered size “was” nothing, because the button is content-visibility: hidden , remember? That means we need to render it if only for a single millisecond, and that’s what this animation does:

@keyframes show-content { from { content-visibility: visible; } }

button { /* Hide it by default */ &:not([hidden=“until-found”]) { content-visibility: hidden; }

/* But make it visible for 1ms */ animation: 1ms show-content;

Software Craftsmanship in the Age of AI

Tim O'Reilly · O'Reilly Radar

On March 26, Addy Osmani and I are hosting the third O'Reilly AI Codecon , and this time we're taking on the question of what software craftsmanship looks like when AI agents are writing much of the code.

The subtitle of this event, "Software Craftsmanship in the Age of AI," was meant to be provocative. Craftsmanship implies care, intention, and deep skill. It implies a maker who touches the material. But we're entering a world where some people with quite impressive output don't touch the code. Steve Yegge, in our conversation earlier this week , put it bluntly: "Code is a liquid. You spray it through hoses. You don't freaking look at it." Wes McKinney, the creator of pandas and one of our speakers at this event, doesn't write code by hand any more either . He's burning north of 10 billion tokens a month across Claude, Codex, and Gemini, writing vast amounts of Go, a language he's never coded in manually.

If that's where this is headed, then what exactly are we crafting? That's the question this lineup is built to answer, and the speakers come at it from very different angles.

The "dark factory" position

One end of the spectrum is occupied by people who are already operating what are increasingly being called dark factories , after the robot factories where there are no lights because the robots that do all of the work don't need them. These are software production environments where humans set direction but agents do nearly all the implementation.

Ryan Carson is the clearest example on our stage. Ryan built and sold Treehouse, where he helped over a million people learn to code. Now he's building Antfarm , an open source tool that lets you install an entire team of agents into OpenClaw with a single command. His talk, "How to Create a Team of Agents in OpenClaw and Ship Code with One Command," is essentially a tutorial on running a software factory where a planning agent decomposes your feature request into user stories, each story gets implemented and tested in isolation by a separate agent, failures retry automatically, and you get back tested pull requests. This isn't quite a dark factory, though. Ryan has built a CI pipeline where the agent records itself using a feature and attaches the video to the PR for human review. It's an assembly line, and the human's job is to inspect the output, not produce it.

This is Steve Yegge's Level 7 or 8 , and it's no longer theoretical. But Ryan's talk will also reveal what happens at the edges, when agents break, when the feedback loop fails, when automated retries aren't enough.

The craftsmanship-means-oversight position

At the other end you have people who are deeply enthusiastic about AI coding but insist that the human role isn't just "set direction and walk away." It's active, continuous, and skilled.

Addy Osmani anchors this position. His talk, "Orchestrating Coding Agents: Patterns for Coordinating Agents in Real-World Software Workflows," is about the coordination problem. As he and I discussed in our recent conversation , there's a spectrum from solo founders running hundreds of agents without reviewing the code to enterprise teams with quality gates and long-term maintenance to think about. Most real teams are somewhere in the middle, and they need patterns, not just tools. Addy has been thinking hard about what Andrej Karpathy called "context engineering," the discipline of structuring all the information an LLM needs to perform reliably. His new book Beyond Vibe Coding is essentially a manual for this new discipline.

Cat Wu from Anthropic brings the platform maker's perspective. She leads product for Claude Code and Cowork, and her focus on building AI systems that are "reliable, interpretable, and steerable" represents a design philosophy that the tool should make human oversight natural and easy. Where Ryan Carson's approach pushes toward maximum agent autonomy, Cat's work at Anthropic is about giving humans the right levers to stay meaningfully in the loop. I'm really looking forward to the conversation between Cat and Addy.

The costs of getting it wrong

Several speakers are focused squarely on what happens when the dark factory breaks down.

Nicole Koenigstein's talk, "The Hidden Cost of Agentic Failure and the Next Phase of Agentic AI," is about the failure modes that don't show up in demos. Nicole is writing the O'Reilly book AI Agents: The Definitive Guide , and she's been consulting with companies on the gap between what agents can do in a sandbox and what they do in production. Hila Fox from Qodo brings a complementary perspective with "From Prompt to Multi-Agent System: The Evolution of Our AI Product," which traces the real path from a simple prompt-based tool to a production multi-agent system, including all the things that go wrong along the way.

The lightning talks share more results of real-world experience. Advait Patel , a site reliability engineer at Broadcom, will talk about what happens when AI agents break production systems, and how his team responded. Abhimanyu Anand from Elastic asks a question that should keep every AI builder up at night: "Is your eval lying to you?" If your evaluation framework is giving you false confidence, you're building on sand.

The bottleneck was never hands on keyboards

Wes McKinney's talk, "The Mythical Agent-Month," revisits Fred Brooks's famous argument that adding more people to a late software project makes it later, and asks whether the same dynamics apply to adding more agents. Wes's answer, as he's laid it out in his blog post , is so compelling that we

Generative AI in the Real World: Sharon Zhou on Post-Training

Ben Lorica and Sharon Zhou · O'Reilly Radar

Post-training gets your model to behave the way you want it to. As AMD VP of AI Sharon Zhou explains to Ben on this episode, the frontier labs are convinced, but the average developer is still figuring out how post-training works under the hood and why they should care. In their focused discussion, Sharon and Ben get into the process and trade-offs, techniques like supervised fine-tuning, reinforcement learning, in-context learning, and RAG, and why we still need post-training in the age of agents. (It's how to get the agent to actually work.) Check it out.

About the Generative AI in the Real World podcast: In 2023, ChatGPT put AI on everyone's agenda. In 2026, the challenge will be turning those agendas into reality. In Generative AI in the Real World, Ben Lorica interviews leaders who are building with AI. Learn from their experience to help put AI to work in your enterprise.

This transcript was created with the help of AI and has been lightly edited for clarity.

00.00 Today we have a VP of AI at AMD and old friend, Sharon Zhou. And we're going to talk about post-training mainly. But obviously we'll cover other topics of interest in AI. So Sharon, welcome to the podcast.

00.17 Thanks so much for having me, Ben.

00.19 All right. So post-training. . . For our listeners, let's start at the very basics here. Give us your one- to four-sentence definition of what post-training is even at a high level?

00.35 Yeah, at a high level, post-training is a type of training of a language model that gets it to behave in the way that you want it to. For example, getting the model to chat, like the chat in ChatGPT was done by post-training.

So basically teaching the model to not just have a huge amount of knowledge but actually be able to have a dialogue with you, for it to use tools, hit APIs, use reasoning and think through things step-by-step before giving an answer—a more accurate answer, hopefully. So post-training really makes the models usable. And not just a piece of raw intelligence, but more, I would say, usable intelligence and practical intelligence.

01.14 So we're two or three years into this generative AI era. Do you think at this point, Sharon, you still need to convince people that they should do post-training, or that's done; they're already convinced?

01.31 Oh, they're already convinced because I think the biggest shift in generative AI was caused by post-training ChatGPT. The reason why ChatGPT was amazing was actually not because of pretraining or getting all that information into ChatGPT. It was about making it usable so that you could actually chat with it, right?

So the frontier labs are doing a ton of post-training. Now, in terms of convincing, I would say that for the frontier labs, the new labs, they don't need any convincing for post-training. But I think for the average developer, there is, you know, something to think about on post-training. There are trade-offs, right? So I think it's really important to learn about the process because then you can actually understand where the future is going with these frontier models.

02.15 But I think there is a question of how much you should do on your own, versus, us[ing] the existing tools that are out there.

02.23 So by convincing, I mean not the frontier labs or even the tech-forward companies but your mom and pop. . . Not mom and pop. . . I guess your regular enterprise, right?

At this point, I'm assuming they already know that the models are great, but they may not be quite usable off the shelf for their very specific enterprise application or workflow. So is that really what's driving the interest right now—that people are actually trying to use these models off the shelf, and they can't make them work off the shelf?

03.04 Well, I was hoping to be able to talk about my neighborhood pizza store post-training. But I think, actually, for your average enterprise, my recommendation is less so trying to do a lot of the post-training on your own—because there's a lot of infrastructure work to do at scale to run on a ton of GPUs, for example, in a very stable way, and to be able to iterate very effectively.

I think it's important to learn about this process, however, because I think there are a lot of ways to influence post-training so that your end objective can happen in these frontier models or inside of an open model, for example, to work with people who have that infrastructure set up. So some examples could include: You could design your own RL environment, and what that is is a little sandbox environment for the model to go learn a new type of skill—for example, learning to code. This is how the model learns to code or learns math, for example. And it's a little environment that you're able to set up and design. And then you can give that to the different model providers or, for example, APIs can help you with post-training these models. And I think that's really valuable because that gets the capabilities into the model that you want, that you care about at the end of the day.

04.19 So a few years ago, there was this general excitement about supervised fine-tuning. And then suddenly there were all these services that made it dead simple. All you had to do is come up with labeled examples. Granted, that that can get tedious, right? But once you do that, you upload your labeled examples, go out to lunch, come back, you have an endpoint that's fine-trained, fine-tuned. So what happened to that? Is that something that people ended up continuing down that path, or are they abandoning it, or are they still using it but with other things?

Keep Deterministic Work Deterministic

Andrew Stellman · O'Reilly Radar

This is the second article in a series on agentic engineering and AI-driven development. Read part one here , and look for the next article on April 2 on O'Reilly Radar.

The first 90 percent of the code accounts for the first 90 percent of the development time. The remaining 10 percent of the code accounts for the other 90 percent of the development time. — Tom Cargill, Bell Labs

One of the experiments I've been running as part of my work on agentic engineering and AI-driven development is a blackjack simulation where an LLM plays hundreds of hands against blackjack strategies written in plain English. The AI uses those strategy descriptions to decide how to make hit/stand/double-down decisions for each hand, while deterministic code deals the cards, checks the math, and verifies that the rules were followed correctly.

Early runs of my simulation had a 37% pass rate. The LLM would add up card totals wrong, skip the dealer's turn entirely, or ignore the strategy it was supposed to follow. The big problem was that these errors compounded: If the model miscounted the player's total on the third card, every decision after that was based on a wrong number, so the whole hand was garbage even if the rest of the logic was fine.

There's a useful way to think about reliability problems like that: the March of Nines . Getting an LLM-based system to 90% reliability is the first nine, and it's the "easy" one. Getting from 90% to 99% takes roughly the same amount of engineering effort. So does getting from 99% to 99.9%. Each nine costs about as much as the last, and you never stop marching. Andrej Karpathy coined the term from his experience building self-driving systems at Tesla, where they spent years earning two or three nines and still had more to go.

Here's a small exercise that shows how that kind of failure compounding works. Open any AI chatbot running an early 2026 model (I used ChatGPT 5.3 Instant) and paste the following eight prompts one at a time, each in a separate message. Go ahead, I'll wait.

Prompt 1: Track a running "score" through a 7-step game. Do not use code, Python, or tools. Do this entirely in your head. For each step, I will give you a sentence and a rule.

CRITICAL INSTRUCTION: You must reply with ONLY the mathematical equation showing how you updated the score. Example format: 10 + 5 = 15 or 20 / 2 = 10. Do not list the words you counted, do not explain your reasoning, and do not write any other text. Just the equation.

Start with a score of 10. I'll give you the first step in the next prompt.

Prompt 2: "The sudden blizzard chilled the small village communities." Add the number of words containing double letters (two of the exact same letter back-to-back, like 'tt' or 'mm').

Prompt 3: "The clever engineer needed seven perfect pieces of cheese." If your score is ODD, add the number of words that contain EXACTLY two 'e's. If your score is EVEN, subtract the number of words that contain EXACTLY two 'e's. (Do not count words with one, three, or zero 'e's).

Prompt 4: "The good sailor joined the eager crew aboard the wooden boat." If your score is greater than 10, subtract the number of words containing consecutive vowels (two different or identical vowels back-to-back, like 'ea', 'oo', or 'oi'). If your score is 10 or less, multiply your score by this number.

Prompt 5: "The quick brown fox jumps over the lazy dog." Add the number of words where the THIRD letter is a vowel (a, e, i, o, u).

Prompt 6: "Three brave kings stand under black skies." If your score is an ODD number, subtract the number of words that have exactly 5 letters. If your score is an EVEN number, multiply your score by the number of words that have exactly 5 letters.

Prompt 7: "Look down, you shy owl, go fly away." Subtract the number of words that contain NONE of these letters: a, e, or i.

Prompt 8: "Green apples fall from tall trees." If your score is greater than 15, subtract the number of words containing the letter 'a'. If your score is 15 or less, add the number of words containing the letter 'l'.

The exercise tracks a running score through seven steps. Each step gives the model a sentence and a counting rule, and the score carries forward. The correct final score is 60 . Here's the answer key: start at 10, then 16 (10+6), 12 (16−4), 5 (12−7), 10 (5+5), 70 (10×7), 63 (70−7), 60 (63−3).

I ran this twice at the same time (using ChatGPT 5.3 Instant), and got two completely different wrong answers the first time I tried it. Neither run reached the correct score of 60:

Step Correct Run 1 (transcript) Run 2 (transcript) 1. Double letters 10 + 6 = 16 10 + 2 = 12 10 + 5 = 15 2. Exactly two 'e's 16 − 4 = 12 12 − 4 = 8 15 + 4 = 19 3. Consecutive vowels 12 − 7 = 5 8 × 7 = 56 19 − 5 = 14 4. Third letter vowel 5 + 5 = 10 56 + 5 = 61 14 + 3 = 17 5. Exactly 5 letters 10 × 7 = 70 61 − 7 = 54 17 − 4 = 13 6. No a, e, or i 70 − 7 = 63 54 − 7 = 47 13 − 3 = 10 7. Words with 'a' or 'i' 63 − 3 = 60 47 − 3 = 44 10 + 4 = 14

The two runs tell very different stories. In Run 1, the model miscounted in Step 1 (found 2 double-letter words instead of 6) but actually got the later counts right. It didn't matter. The wrong score in Step 1 flipped a branch in Step 3, triggering a multiply instead of a subtract, and the score never recovered. One early mistake threw off the entire chain, even though the model was doing good work after that.

Should you adopt Java 12 or stick on Java 11?

Stephen Colebourne · Stephen Colebourne (Joda)

Should you adopt Java 12 or stick on Java 11 for the next 3 years? Seems like an innocuous question, but it is one of the most important decisions out there for those running on the JVM. I'll try to cover the key aspects of the decision, with the assumption that you care about running with the latest set of security patches in production.

TL;DR, It is vital to fully understand and accept the risks before adopting Java 12.

The Java release train

There is now a new release of Java every six months, so Java 12 is less than five months away despite Java 11 having just been released. As part of the process of moving to more frequent releases, certain releases are designated to be LTS (long term support) and as such will have security patches available for four years or more. This makes them "major" releases, not because they have a bigger feature set but because they have multi year support.

It is expected that Java 11 patches (11.0.1, 11.0.2, 11.0.3, etc.) will be smaller and simpler than Java 8 updates (8u20, 8u40, 8u60, etc.) - Java 11 updates will be more focused on security patches, without the internal enhancements of Java 8 updates. Instead, Oracle want us to think of Java 12, 13, 14 etc. as small upgrades, similar to an imaginary Java 11u20, 11u40 etc. To be blunt, I find this nonsensical.

Senior Oracle employees have repeatedly argued that updates such as 8u20 and 8u40 often broke code. This was not my experience. In fact my experience was that update releases primarily contained bug fixes. The only break I can remember was the addition of –allow-script-in-comments to Javadoc, which isn't a core part of Java. As a result, I have never feared picking up the latest update release - and this has been a core benefit of the Java platform.

Drilling down into why update releases tend to cause no problems, lets examine the differences between release types:

Model Old model New model

Upgrade Java major releases Java update releases Java release train Java patches

Frequency Every 3 years or so Every 6 months Every 6 months Every 3 months

Versions 6 -> 7 -> 8 8 -> 8u20 -> 8u40 11 -> 12 -> 13 11 -> 11.0.1 -> 11.0.2

Language changes ☒ ✘ ☒ ✘

JVM changes ☒ ✘ ☒ ✘

Major enhancements ☒ ✘ ☒ ✘

Added classes/methods ☒ ✘ ☒ ✘

Removed classes/methods ✘ ✘ ☒ ✘

New deprecations ☒ ✘ ☒ ✘

Internal enhancements ☒ ☒ ☒ ✘

JDK tool changes ☒ ☒ ☒ ✘

Bug fixes ☒ ☒ ☒ ☒

Security patches ☒ ☒ ☒ ☒

Given the table above, I find it amazing that anyone would claim moving from 11 to 12 to 13 is anything like moving from 8u20 to 8u40. But that is the official Oracle viewpoint:

Oracle FAQ

As the table clearly shows, each version in the Java release train can contain any change traditionally associated with a full major version. These include language changes and JVM changes, both of which have major impacts on IDEs, bytecode libraries and frameworks. In addition, there will not only be additional APIs, but also API removals (something that did not happen prior to 8).

Oracle's claim is that because each release is only 6 months after the previous one, there won't be as much "stuff" in it, thus it won't be as hard to upgrade. While true, it is also irrelevant. What matters is whether the upgrade has the potential to damage your code stack or not. And clearly, going from 11 -> 12 -> 13 has much greater potential for damage than 8 -> 8u20 -> 8u40 ever did .

The key difference compared to updates like 8u20 -> 8u40 are the changes to the bytecode version, and the changes to the specification. Changes to the bytecode version tend to be particularly disruptive, with most frameworks making heavy use of libraries like ASM or ByteBuddy that are intimately linked to each bytecode version. And moving from 8u20 -> 8u40 still had the same Java SE specification, with all the same classes and methods, something that cannot be relied on moving from 12 to 13. I simply do not accept Oracle's argument that the "amount of stuff" in a release is more significant than the "type of stuff".

Note however that another one of Oracle's claims really does matter. They point out that if you stick with Java 11 and plan to move to the next LTS version when it is released (ie. Java 17) that you might find your code doesn't compile. Remember that the Java development rules now state that an API method can be deprecated in one version and removed in the next one. Rules that do not take LTS releases into account. So, a method could be deprecated in 13 and removed in 15. Someone upgrading from 11 to 17 would simply find a deleted API having having never seen the deprecation. Lets not panic too much about removal though - the only APIs likely to be removed are specialist ones, not those in widespread use by application code.

Considerations before adopting the release train

In this section, I try to outline some of the considerations/

Java switch - 4 wrongs don’t make a right

Stephen Colebourne · Stephen Colebourne (Joda)

The switch statement in Java is being changed. But is it an upgrade or a mess?

Classic switch

The classic switch statement in Java isn’t great. Unlike many other parts of Java, it wasn’t properly rethought when pulling features across from C all those years ago.

The key flaw is “fall-through-by-default”. This means that if you forget to put a break clause in each case , processing will continue on to the next case clause.

Another flaw is that variables are scoped to the entire switch, thus you cannot reuse a variable name in two different case clauses. In addition, default clause is not required, which leaves readers of the code unclear as to whether a clause was forgotten or not.

And of course there is also the key limitation - that the type to be switched on can only be an integer, enum or string.

String instruction; switch (trafficLight) { case RED: instruction = “Stop”; case YELLOW: instruction = “Prepare”; break; case GREEN: instruction = “Go”; break; } System.out.println(instruction);

The code above does not compile because there is no default clause, leaving instruction undefined. But even if it did compile, it would never print “Stop” due to the missing break . In my own coding, I prefer to always put a switch at the end of a method, with each clause containing a return to reduce the risks of switch.

Upgraded switch

As part of Project Amber, switch is being upgraded . But sadly, I’m unconvinced as to the merits of the new design. To be clear, there are some good aspects, but overall I think the solution is overly complex and with some unpleasant syntax choices.

The key aim is to add an expression form, where you can assign the result of the switch to a variable. This is rather like the ternary operator (eg. x != null ? x : “”), which is the expression equivalent of an if statement. An expression form would reduce problems like the undefined variable above, because it makes it more obvious that each branch must result in a variable.

The current plan is to add not one, but three new forms of switch. Yes, three.

Explaining this in a blog post is, unsurprisingly, going to take a while...

Type 1: Statement with classic syntax. As today. With fall-through-by-default. Not exhaustive.

Type 2: Expression with classic syntax. NEW! With fall-through-by-default. Must be exhaustive.

Type 3: Statement with new syntax. NEW! No fall-through. Not exhaustive.

Type 4: Expression with new syntax. NEW! No fall-through. Must be exhaustive.

The headline example (type 4) is of course quite nice:

As can be seen, the new syntax of type 3 and 4 uses an arrow instead of a colon. And there is no need to use break if the code consists of a single expression. There is also no need for a default clause when using an enum, because the compiler can insert it for you provided you’ve included all the known enum values. So, if you missed out GREEN , you would get a compile error.

The devil of course is in the detail.

Firstly, a clear positive. Instead of falling through by listing multiple labels, they can be comma-separated:

Straightforward and obvious. And avoids many of the simple fall-through use cases.

What if the code to execute is more complex than an expression?

yield ? Shrug. For a long time it was going to be break {expression} , but this clashes with labelled break (a syntax feature that is rarely used).

So what about type 2?

/\/ type 2 var instruction = switch (trafficLight) { case RED: yield “Stop”; case YELLOW: System.out.println(“Prepare”); case GREEN: yield “Go”; }; System.out.println(instruction);

Oops! I forgot the yield . So, an input of YELLOW will output “Prepare” and then fall-through to yield “Go”.

So, why is it proposed to add a new form of switch that repeats the fall-through-by-default error from 20 years ago? The answer is orthogonality - a 2x2 grid with expression vs statement and fall-through-by-default vs no fall-through.

A key question is whether being orthogonal justifies adding a almost totally useless form of switch (type 2) to the language.

> *This is rather like the ternary operator (eg.*
>
> Stephen Colebourne

So, type 3 is fine them?

Well, no. Because of the insistence on orthogonality, and thus an insistence of copying the historic rules relating to type 1 statement switch, there is no requirement to list all

In Praise of –dry-run

Henrik Warne

For the last few months, I have been developing a new reporting application. Early on, I decided to add a –dry-run option to the run command. This turned out to be quite useful – I have used it many times a day while developing and testing the application.

The application will generate a set of reports every weekday. It has a loop that checks periodically if it is time to generate new reports. If so, it will read data from a database, apply some logic to create the reports, zip the reports, upload them to an sftp server, check for error responses on the sftp server, parse the error responses, and send out notification mails. The files (the generated reports, and the downloaded feedback files) are moved to different directories depending on the step in the process. A simple and straightforward application.

Early in the development process, when testing the incomplete application, I remembered that Subversion (the version control system after CVS, before Git) had a –dry-run option. Other linux commands have this option too. If a command is run with the argument –dry-run , the output will print what will happen when the command is run, but no changes will be made. This lets the user see what will happen if the command is run without the –dry-run argument.

I remembered how helpful that was, so I decided to add it to my command as well. When I run the command with –dry-run , it prints out the steps that will be taken in each phase: which reports that will be generated (and which will not be), which files will be zipped and moved, which files will be uploaded to the sftp server, and which files will be downloaded from it (it logs on and lists the files).

Looking back at the project, I realized that I ended up using the –dry-run option pretty much every day.

I am surprised how useful I found it to be. I often used it as a check before getting started. Since I know –dry-run will not change anything, it is safe to run without thinking. I can immediately see that everything is accessible, that the configuration is correct, and that the state is as expected. It is a quick and easy sanity check.

I also used it quite a bit when testing the complete system. For example, if I changed a date in the report state file (the date for the last successful report of a given type), I could immediately see from the output whether it would now be generated or not. Without –dry-run , the actual report would also be generated, which takes some time. So I can test the behavior, and receive very quick feedback.

The downside is that the dryRun -flag pollutes the code a bit. In all the major phases, I need to check if the flag is set, and only print the action that will be taken, but not actually doing it. However, this doesn't go very deep. For example, none of the code that actually generates the report needs to check it. I only need to check if that code should be invoked in the first place.

The type of application I have been writing is ideal for –dry-run . It is invoked by a command, and it may create some changes, for example generating new reports. More reactive applications (that wait for messages before acting) don't seem to be a good fit.

I added –dry-run on a whim early on in the project. I was surprised at how useful I found it to be. Adding it early was also good, since I got the benefit of it while developing more functionality.

The –dry-run flag is not for every situation, but when it fits, it can be quite useful.

Downside The downside is that the dryRun -flag pollutes the code a bit.

Henrik Warne

Solid Relevance

Robert C Martin (Clean Coder)

Recently I received a letter from someone with a concern. It went like this:

For years the knowledge of the SOLID principle has been a standard part of our recruiting procedure. Candidates were expected to have a good working knowledge of these principles. Lately, however, one of our managers, who doesn't code much anymore, has questioned whether that is wise. His points were that the Open-Closed principle isn't very important anymore because most of the code we write isn't contained in large monoliths and making changes to small microservices is safe and easy. The Liskov Substitution Principle is long out of date because we don't focus on inheritance nearly as much as we did 20 years ago. I think we should consider Dan North's position on SOLID – "Just write simple code."

I wrote the following letter in response:

The SOLID principles remain as relevant to day as they were in the 90s (and indeed before that). This is because software hasn't changed all that much in all those years — and that is because software hasn't change all that much since 1945 when Turing wrote the first lines of code for an electronic computer. Software is still if statements, while loops, and assignment statements — Sequence , Selection , and Iteration .

Every new generation likes to think that their world is vastly different from the generation before. Every new generation is wrong about that; which is something that every new generation learns once the next new generation comes along to tell them how much everything has changed.

So let's walk through the principles, one by one.

SRP) The Single Responsibility Principle.

Gather together the things that change for the same reasons. Separate things that change for different reasons.

It is hard to imagine that this principle is not relevant in software. We do not mix business rules with GUI code. We do not mix SQL queries with communications protocols. We keep code that is changed for different reasons separate so that changes to one part to not break other parts. We make sure that modules that change for different reasons do not have dependencies that tangle them.

Microservices do not solve this problem. You can create a tangled microservice, or a tangled set of microservices if you mix code that changes for different reasons.

Dan North's slides completely miss the point on this, and convinces me that he did not understand the principle at all. (or that he was being ironic, which knowing Dan, is far more likely) His answer to the SRP is to "Write Simple Code". I agree. The SRP is one of the ways we keep the code simple.

OCP) The Open-Closed Principle.

A Module should be open for extension but closed for modification.

Of all the principles, the idea that anyone would question this one fills me full of dread for the future of our industry. Of course we want to create modules that can be extended without modifying them. Can you imagine working in a system that did not have device independence, where writing to a disk file was fundamentally different than writing to a printer, or a screen, or a pipe? Do we want to see if statement scattered through our code to deal with all the little details?

Or... Do we want to separate abstract concepts from detailed concepts. Do we want to keep business rules isolated from the nasty little details of the GUI, and the micro-service communications protocols, and the arbitrary behaviors of the database? Of course we do!

Again, Dan's slide gets this completely wrong. When requirements change only part of the existing code is wrong. Much of the existing code is still right. And we want to make sure that we don't have to change the right code just to make the wrong code work again. Dan's answer is "write simple code". Again, I agree. And, ironically, he is right. Simple code is both open and closed .

LSP) The Liskov Substitution Principle.

A program that uses an interface must not be confused by an implementation of that interface.

People (including me) have made the mistake that this is about inheritance. It is not. It is about sub-typing. All implementations of interfaces are subtypes of an interface. All duck-types are subtypes of an implied interface. And, every user of the base interface, whether declared or implied, must agree on the meaning of that interface. If an implementation confuses the user of the base type, then if/switch statements will proliferate.

This principle is about keeping abstractions crisp and well-defined. It is impossible to believe that this is an outmoded concept.

Dan's slides are entirely correct on this topic; he simply missed the point of the principle. Simple code is code that maintains crisp subtype relationships.

ISP) The Interface Segregation Principle.

Keep interfaces small so that users don't end up depending on things they don't need.

We still work with compiled languages. We still depend upon modification dates to determine which modules should be recompiled and redeployed. So long as this is true we will have to face the problem that when module A depends on module B at compile time, but not at run time, then changes to module B will force recompilation and

Passing planes and other whoosh sounds

Oona Räisänen · Oona Raisanen (windytan)

I always assumed that the recognisable 'whoosh' sound a plane or helicopter makes when passing overhead simply comes from the famous Doppler effect . But when you listen closely, this explanation doesn't make complete sense.

(Audio clipped from freesound - here and here)

A classic example of the Doppler effect is the sound of a passing ambulance constantly descending in pitch. When a plane flies overhead the roar of the engine sometimes does that as well. But you can also hear a wider, breathier noise that does something different: it's like the pitch goes down at first, but when the plane has passed us, the pitch goes up again. That's not how Doppler works! What's going on there?

Comb filtering.

Let's shed light on the mystery by taking a look at the sound in a time-frequency spectrogram. Here, time runs from left to right, frequencies from top (high) to bottom (low).

We can clearly see one part of the sound (the one starting at 2500 Hz) sweeping downwards, or from high to low frequencies; this should be the Doppler effect. But there's something else happening at the bottom of the image – another pattern is moving down at first, then up again...

This sound's frequency distribution seems to form a series of moving peaks and valleys. This resembles what audio engineers would call 'comb filtering' , due to its appearance in the spectrogram. When the peaks and valleys move about it causes a 'whoosh' sound; this is the same principle as in the flanger effect used in music production. But these are just jargon for the electronically created version. We can call the acoustic phenomenon the whoosh.

The comb pattern is caused by two copies of the same exact sound arriving at a slightly different times, close enough that they form an interference pattern. It's closely related to what happens to light in the double slit experiment . In recordings this often means that the sound was captured by two microphones and then mixed together; you can sometimes hear this happen unintentionally in podcasts and radio shows. So my thought process is, are we hearing two copies of the plane's sound? How much later is the other one arriving, and why? And why does the 'whoosh' appear to go down in pitch at first, then up again?

Into the cepstral domain.

The cepstrum , which is the inverse Fourier transform of the estimated log spectrum, is a fascinating plot for looking at delays and echoes in complex (as in complicated) signals. While the spectrum separates frequencies, the cepstrum measures time, or quefrency – see what they did there? It reveals cyclicities in the sound's structure even if it interferes with itself, like in our case. In that it's similar to autocorrelation.

It's also useful for looking at sounds that, experientially, have a 'pitch' to them but that don't show any clear spectral peak in the Fourier transform. Just like the sound we're interested in.

Here's a time- quefrency cepstrogram of the same sound (to be accurate, I used the autocepstrum here for better clarity):

The Doppler effect is less prominent here. Instead, the plot shows a sweeping peak that seems to agree with the pitch change we hear. This delay time sweeps from around 4 milliseconds to 9 ms and back. Note that the scale: higher frequencies (shorter times) are on the bottom this time.

Now why would the sound be so correlated with itself with this sweeping delay time?

Ground echo?

Here's my hypothesis. We are hearing not only the direct sound from the plane but also a delayed echo from a nearby flat surface. These two sound get superimposed and interfere before they reach our ears. The effect would be especially prominent with planes and helicopters because there is little in the way of the sound either from above or from the large surface. And what could be a large reflective surface outdoors? Well, the ground below!

Let's think about the numbers. The ground is around one-and-a-half metres below our ears. When a plane is directly overhead, the reflected sound needs to take a path that's three metres longer (two-way) than the direct path. Since sound travels 343 metres per second this translates to a difference of 9 milliseconds – just what we saw in the correlogram!

Below, I used GeoGebra to calculate the time difference (between the yellow and green paths) in milliseconds.

When the plane is far away the angle is shallower, the two paths are more similar in distance, and the time difference is shorter.

It would follow that a taller person hears the sound differently than a shorter one, or someone in a tenth-floor window! If the ground is very soft, maybe in a mossy grove, you probably wouldn't hear the effect at all; just the Doppler effect. But this prediction needs to be tested out in a real forest.

Here's what a minimal acoustic simulation model renders. We'll just put a flying white noise source in the sky and a reflective surface as the ground. Let's only update the IR at 15 fps to prevent the Doppler phenomenon from emerging.

Whoosh!

Some everyday whooshes.

The whoosh isn't only associated with planes. When it occurs naturally it usually needs three things:

Big problems at the timezone database

Stephen Colebourne · Stephen Colebourne (Joda)

The last time I wrote about the timezone database on this blog, the database was under threat from a lawsuit. Fortunately that lawsuit went away relatively quickly as the company involved got the message that their action was a big mistake. Unfortunately this time the mess is internal.

Paul Eggert is the project lead of the timezone database hosted at IANA, a position referred to as the TZ Coordinator. He is an expert in the field, having been involved in documenting timezone data for decades. Unfortunately, he is currently ignoring all objections to an action only he seems intent on making to solve an invented problem that only he sees as important.

The database is the world's principle source of timezone information. The data is included in everything from operating systems to smartphones to programming language development kits such as the JDK. While you may never have heard of it, the sheer pervasiveness of the data makes the potential impact of change or damage pretty huge.

The timezone database contains information about how clocks have varies in each region around the world. The mandate of the project is to record this information from 1970 onwards.

Of course, computers being what they are, a function that returns the timezone for a given date can be passed in a pre-1970 date as well as a post-1970 one. For this, and reasons of completeness, the timezone database contains pre-1970 data as well as post-1970 data. If you go to your JDK or operating system and ask for the timezone offset for 1920-01-01 for the ID "Europe/Oslo" or "Europe/Berlin" you will get an answer:

DateTimeZone oslo = DateTimeZone.forID("Europe/Oslo"); System.out.println(oslo.getOffset(new DateTime(1948, 6, 1, 12, 0))); //prints 3600000 DateTimeZone berlin = DateTimeZone.forID("Europe/Berlin"); System.out.println(berlin.getOffset(new DateTime(1948, 6, 1, 12, 0))); //prints 7200000

The proposed change is to downgrade "Europe/Oslo" to be merely an alias for "Europe/Berlin". The rationale is that since the two regions have the same data post-1970 there should only be one ID. The issue with this is that querying "Europe/Oslo" in pre-1970 (as above) will now return the data from Berlin. ie. the well researched pre-1970 data for Oslo will be replaced by that of Berlin.

The situation with Joda-Time is even worse. With Joda-Time, aliases (also known as Links) are actively resolved. Before the proposed change this test case passes, after the proposed change the test case fails:

assertEquals("Europe/Oslo", DateTimeZone.forID("Europe/Oslo").getID());

In other words, it will be impossible for a Joda-Time user to hold the ID "Europe/Oslo" in memory. This could be pretty catastrophic for systems that rely on timezone management, particularly ones that end up storing that data in a database. To mitigate this to some degree, I've added a test case and released a version that has such a test case in it, but obviously users of Joda-Time that haven't upgraded to the latest version may still see problems if they update the tzdb version.

But what about backwards compatibility? Well it seems that the TZ Coordinator just doesn't see this as being important. To him, it doesn't matter that users may be relying on this data.

(Technically, the data has moved, not been deleted. But the file containing the moved data is never normally used by downstream systems, thus to all intents and purposes it has been deleted.)

The question you might ask is why is Berlin favoured over Oslo? Why can Berlin keep its status and full history, but Oslo gets effectively deleted? The answer is that Berlin has the greater population - would you have guessed that if you didn't read it here?

From my perspective, I cannot see how it is not incredibly unfair that the timezone ID that represents the country of Norway, "Europe/Oslo", is treated as less important than the ID that represents the country of Germany, "Europe/Berlin". Yes, there is a technical rationale around 1970 and population, but that is really pretty arcane.

In fact it goes further than this. The TZ Coordinator does not really believe that there should be an ID for Oslo/Norway at all. (The official project rules say that there should only be one ID for locations where timezone data is the same post-1970. Country borders simply dont matter.)

Some of the 30 IDs proposed for downgrade are "Europe/Oslo", "Europe/Stockholm", "Europe/Copenhagen", "Europe/Amsterdam", "Europe/Luxembourg", "Europe/Monaco", "Atlantic/Reykjavik" and "Indian/Mahe". If you regularly use any of these IDs you may be affected by this change. Iceland is a classic case - "Atlantic/Reykjavik" is to be downgraded in favour of "Africa/Abidjan". Bonus points if you know which country Abidjan is in!

What is driving the change? Well this is where it gets really weird. The TZ Coordinator's argument is that there is a fairness/equity problem if Oslo is allowed to keep its pre-1970 history but other locations (typically in Africa) are not. I have two problems with this. Firstly, I consider it to also be unfair and inequitable that Berlin gets to have pre-1970 history and Oslo does not. Secondly, the correct approach to solving a fairness/equity problem is to level up, not level down. ie. Most leaders would want to improve the worse performers on the team, not force the best performers to be as bad as the worst.

So, we have a change with terrible downstream effects,

Programming Conference – Jfokus Stockholm 2025

Henrik Warne

This week I attended the Jfokus software development conference in Stockholm, Sweden. I first went in 2011, and I have been back many times through the years. The conference has a Java focus (duh!), but many talks cover general topics as well.

The whole development team at NGM got tickets. It is really nice to be able to discuss and compare notes with your colleagues. The big theme this year, apart from Java, was of course AI and LLMs.

Talks I Liked

The First 80% of Reading One Billion Rows Fast Enough by René Schwietzke . This is a talk on Java optimization, and I really enjoyed it! I had not heard about the challenge before. The input is one billion rows of simple weather csv data, and the idea is to see how fast it can be processed in Java. Of course there are solutions that use incredibly weird and obscure techniques, but this talk goes through some basic optimizations that together add up to a run time of 20% of the original solution.

After setting up the problem, René shows a base-line solution, and gives its run time. Then he goes through a number of optimizations, and shows how much each saves. Examples of optimizations are: replace split() by indexOf(), use int instead of double (we know there is only ever one decimal digit), mutate existing objects instead of creating new ones, only read bytes (not Strings, doubles etc) and delay Unicode processing, simpler Min/Max, create the hash code while traversing the line.

René used a flamegraph from a profiler to guide what areas of the program should be optimized. Some general rules he followed were: replace standard JDK functionality (it may be more general than what is needed), low or no memory allocation, avoid wrappers and immutability, reduce branching (if, loops), exploit what you know about the input data.

The Future of Work by Henrik Kniberg . This was second of two keynote talks that opened the conference. Henrik did a live demo where he used several AI agents to accomplish tasks, such as making code changes, creating a branch in git, and creating a PR with the changes. The AI agents have instructions in the form of short text documents, and these instructions can be updated, even by the agents themselves (subject to human approval).

The agents appear as their own users in Slack. They can also be given recurring tasks, for example to create a report each morning on a given subject, and to mail out the report and post a summary of it in Slack. He also demonstrated how they can trouble shoot if something goes wrong, for example if it can't create a git branch. The LLM used in the demo was Claude, and Henrik used it in voice-input mode.

He ended the presentation with some reflections on the implications of this way of working. He, like me, has always loved programming. Will this way of working, with agents writing a large chunk of the code, or all code, mean the end of software development for humans? First he noted that this is similar to moving from punch cards, to assembler, and to compiled higher level languages. You move up one abstraction level. He also noted that what he liked about programming was making and creating things, not necessarily writing the actual code.

A very good and thought-provoking talk. Actually seeing agents in action, instead of merely being told what they can do, really brings the points home.

Ask the Architect with Brian Goetz (Java Language Architect) and Mark Reinhold (Chief Architect), both at Oracle. This was a Q&A session, where the audience had a chance to ask Brian and Mark Java questions. I didn't really know what to expect from this session, since it will depend a lot on the questions asked. But I was pleasantly surprised. There was quite a variety of questions.

There were questions about serialization, GraalVM vs HotSpot, records, streams, Lombok, different deprecation modes, and more. Quite interesting. Before we started, Brian and Mark cautioned us not to begin questions with "Why don't you just...". Modifying a language that has been around for so long, with so many existing programs, is not easy. This became very clear when hearing some of the answers.

The Value of Conferences

Going to a conference is different from watching talks on YouTube, or reading books or blog posts about software development. It is nice to meet and talk to other developers. My standard question when chatting with other attendees is "What is your favorite talk so far, and why?". At Jfokus, your name and your company are printed on your badge. This gives you another set of good icebreaker questions: "What do you do at Company X? What does the company do? What tech stack do you use?". Also, when listening to a talk live, you have a chance to ask questions, either during the talk, or afterwards.

It is very convenient to attend a conference in your home city or country. It is cheaper and there is less travel. From the badges I saw at Jfokus, most people were from Sweden, and a few from Germany.

At Jfokus, there are usually six parallel talks, so it can sometimes be hard to pick what to listen to. When attending a conference, and listening to many talks in a row, you notice things that are mentioned more than once. Examples this year were LLMs checking the output of other LLMs, RAG (Retrieval-Augmented Generation), AI-assisted coding (with IDE plugins), GraalVM, and the Quarkus framework. After a conference I always end up with a long list of things to look up: techniques and tools I hadn't heard about before, and books and articles to look into.

User-defined literals in Java?

Stephen Colebourne · Stephen Colebourne (Joda)

Java has a number of literals for creating values, but wouldn't it be nice if we had more?

Current literals

These are some of the literals we can write in Java today:

integer - 123 , 12s , 1234L , 0xB8E817 , 077 , 0b1011_1010

floating point - 45.6f , 56.7d , 7.656e6

string - "Hello world"

char - 'a'

boolean - true , false

null - null

Project Amber is also considering adding multi-line and/or raw string literals.

But there are many other data types that would benefit from literals, such as dates, regex and URIs.

User-defined literals

In my ideal future, I'd like to see Java extended to support some form of user-defined literals. This would allow the author of a class to provide a mechanism to convert a sequence of characters into an instance of that class. It may be clearer to see some examples using one possible syntax (using backticks):

Currency currency = `GBP`; LocalDate date = `2019-03-29`; Pattern pattern = `strata\.\w+`; URI uri = `https://blog.joda.org/`;

A number of semantic features would be required:

Type inference

Raw processing

Validated at compile-time

Type inference

Type inference is of course a key aspect of literals. It would have to work in a similar way to the existing literals, but with a tweak to handle the new var keyword. ie. these two would be equivalent:

LocalDate date = `2019-03-29`; var date = LocalDate`2019-03-29`;

The type inference would also work with methods (compile error if ambiguous):

boolean inferior = isShortMonth(`2019-04-12`);

public boolean isShortMonth(LocalDate date) { return date.lengthOfMonth()

Raw processing

Processing of the literal should not be limited by Java's escape mechanisms. User-defined literals need access to the raw string. Note that this is especially useful for regex, but would also be useful for files on Windows:

/\/ user-defined literals var pattern = Pattern`strata\.\w+`; /\/ today var pattern = Pattern.compile("strata\\.\\w+");

Today, the `\` needs to be escaped, making the regex difficult to read.

Clearly, the problem with parsing raw literals is that there is no mechanism to escape. But the use cases for user-defined literals tend to have constrained formats, eg. a date doesn't contain random characters. So, although there might be edge cases where this would be a problem, they would vert much be edge cases.

Validated at Compile-time

A key feature of literals is that they are validated at compile-time. You can't use an integer literal to create an int if the value is larger than the maximum allowed integer (2^31).

User-defined literals also need to be parsed and validated at compile-time too. Thus this code would not compile:

LocalDate date = `2019-02-31`;

Most types which would benefit from literals only accept specific input formats, so being able to check this at compile time would be beneficial.

How would it be implemented?

I'm pretty confident that there are various ways it could be done. I'm not going to pick an approach, as ultimately those that control the JVM and language are better placed to decide. Clearly though, there is going to need to be some form of factory method on the user class that performs the parse, with that method invoked by the compiler. And ideally, the results of the parse would be stored in the constant pool rather than re-parsed at runtime.

What I would say is that user-defined literals would almost be a requirement for making value types usable, so something like this may be on the way anyway.

I like literals. And I would really like to be able to define my own!

Any thoughts?

Clearly, the problem with parsing raw literals is that there is no mechanism to escape.

Stephen Colebourne

Java 9 modules - JPMS basics

Stephen Colebourne · Stephen Colebourne (Joda)

The Java Platform Module System (JPMS) is the major new feature of Java SE 9. In this article, I will introduce it, leaving most of my opinions to a follow up article. This is based on these slides .

Java Platform Module System (JPMS)

The new module system, developed as Project Jigsaw , is intended to raise the abstraction level of coding in Java as follows:

The primary goals of this Project are to:

* Make the Java SE Platform, and the JDK, more easily scalable down to small computing devices;

* Improve the security and maintainability of Java SE Platform Implementations in general, and the JDK in particular;

* Enable improved application performance; and

* Make it easier for developers to construct and maintain libraries and large applications, for both the Java SE and EE Platforms.

To achieve these goals we propose to design and implement a standard module system for the Java SE Platform and to apply that system to the Platform itself, and to the JDK. The module system should be powerful enough to modularize the JDK and other large legacy code bases, yet still be approachable by all developers.

However as we shall see, project goals are not always met.

What is a JPMS Module?

JPMS is a change to the Java libraries, language and runtime. This means that it affects the whole stack that developers code with day-to-day, and as such JPMS could have a big impact. For compatibility reasons, most existing code can ignore JPMS in Java SE 9, something that may prove to be very useful.

The key conceptual point to grasp is that JPMS adds new a concept to the JVM - modules. Where previously, code was organized into fields, methods, classes, interfaces and packages, with Java SE 9 there is a new structural element - modules.

a class is a container of fields and methods

a package is a container of classes and interfaces

a module is a container of packages

Because this is a new JVM element, it means the runtime can apply strong access control. With Java 8, a developer can express that the methods of a class cannot be seen by other classes by declaring them private. With Java 9, a developer can express that a package cannot be seen by other modules - ie. a package can be hidden within a module.

Being able to hide packages should in theory be a great benefit for application design. No longer should there be a need for a package to be named “impl” or “internal” with Javadoc declaring “please don’t use types from this package”. Unfortunately, life won’t be quite that simple.

Creating a module is relatively simple however. A module is typically just a jar file that has a module-info.class file at the root - known as a modular jar file . And that file is created from a module-info.java file in your sourcebase (see below for more details).

Using a modular jar file involves adding the jar file to the modulepath instead of the classpath. If a modular jar file is on the classpath, it will not act as a module at all, and the module-info.class will be ignored. As such, while a modular jar file on the modulepath will have hidden packages enforced by the JVM, a modular jar file on the classpath will not have hidden packages at all.

Other module systems

Java has historically had other module systems, most notably OSGi and JBoss Modules. It is important to understand that JPMS has little resemblance to those systems.

Both OSGi and JBoss Modules have to exist without direct support from the JVM, yet still provide some additional support for modules. This is achieved by launching each module in its own class loader, a technique that gets the job done, yet is not without its own issues.

Unsurprisingly, given these are existing module systems, experts from those groups have been included in the formal Expert Group developing JPMS. However, this relationship has not been harmonious. Fundamentally, the JPMS authors (Oracle) have set out to build a JVM extension that can be used for something that can be described as modules, whereas the existing module systems derive experience and value from real use cases and tricky edge cases in big applications that exist today.

> *This is used here as Joda-Beans has methods where the return type is from Joda-Convert.*
>
> Stephen Colebourne

When reading about modules, it is important to consider whether the authors of the article you are reading are from the OSGi/JBoss Modules design camp. (I have never actively used OSGi or JBoss Modules, although I have used Eclipse and other tools that use OSGi internally.)

module-info.java

The module-info.java file contains the instructions that de-

Criminals are renting virtual phones to bypass bank security

Malwarebytes Labs

Researchers at Group-IB warn about criminals using virtual Android devices to bypass modern security solutions.

Cloud phones are virtual Android devices that can fully mimic real device fingerprints (model, hardware, IP, timezone, sensor data, behavior). This allows them to undermine banks' device-based fraud detection.

Originally, phone farms were made up of physical devices and were set up for testing. They grew in number when companies found out they could rent virtual phones and artificially raise engagement stats like follower counts, likes, shares, and so on. Further growth was driven by moving the infrastructure from physical phone farms to cloud phones.

At some point, cybercriminals figured out how to use these "rent-a-phones" to trick people into sharing access to banking accounts and crypto wallets, which were then emptied.

Banks caught on to these tactics and started building mobile apps that rely on device fingerprinting. This helped them detect and block fake devices taking over people's accounts.

But as with any arms race, criminals found a way around that too. They now "pre-warm" devices by adding banking apps, registering credentials, and running small transactions so accounts and device telemetry look low-risk.

The researchers note that:

"They moved to cloud phones—remote-access Android devices running in data centers. For all intents and purposes, these are real phones, running genuine firmware, exhibiting natural sensor behavior, and presenting valid hardware attestation."

And it's not a big investment for the criminals. Major cloud phone platforms offer device rentals for as little as $0.10-0.50 per hour, making fraud infrastructure accessible to almost anyone.

One place these devices are used is in mobile games with real-money economies. These games have long struggled with a specific problem: bot farming of in-game currency and resources. In many cases, automated accounts can generate in-game items that have real-world value.

Banks face a different problem: account take-over (ATO) attacks. As banking shifted from web browsers to mobile apps, they needed more reliable and comprehensive ways to identify trusted devices. Many banks now bind accounts to specific devices and flag transfers that don't come from that device.

The start of an attack is still social engineering. Criminals try to trick users into sharing one-time passwords (OTPs), approve a login, or make a transfer "to a safe account."

Behind the scenes, the criminal logs into a cloud phone instance that already looks like the victim's device to their bank, thanks to matching or plausible fingerprints and pre-warmed behavior.

Once the criminals are in, they carry out authorized push payment (APP) transfers (often to money-mule accounts), that the bank's systems may treat as low-risk because nothing about the device seems obviously wrong.

At that point the criminals can start emptying your account or sell the virtual phones to other criminals. According to the researchers:

"Darknet markets actively trade pre-verified dropper accounts created on cloud phones, with Revolut and Wise accounts priced at $50-200 each, often including continued access to the cloud phone instance."

How to stay safe

The Group-IB researchers advise end users to:

Never complete account verification processes under third-party instruction. Keep in mind that banks and government institutions will not ask customers to authenticate accounts through unfamiliar apps or remote environments.

Enable device-based security features. Use official mobile banking apps, biometric authentication, and strong device-level security settings.

Be cautious of "easy income" schemes involving bank accounts. Fake job offers requiring you to "verify" bank accounts, government officials requesting account verification, bank representatives asking you to move money to "safe" accounts.

If you suspect that you have been targeted, contact your bank immediately. Update passwords and enable multi-factor authentication on all accounts.

We'd like to add:

Turn on banking alerts for logins, payee changes and transactions where possible so you see unusual activity immediately.

Use an up-to-date, real-time anti-malware solution for your Android device to detect and stop information stealers.

When in doubt about a message, consult Malwarebytes Scam Guard . It will help you figure out if it's a scam and guide you through what to do.

We don't just report on phone security—we provide it

Cybersecurity risks should never spread beyond a headline. Keep threats off your mobile devices by downloading Malwarebytes for iOS , and Malwarebytes for Android today.

What OpenClaw Reveals About the Next Phase of AI Agents

Kesha Williams · O'Reilly Radar

In November 2025, Austrian developer Peter Steinberger published a weekend project called Clawdbot. You could text it on Telegram or WhatsApp, and it would do things for you: manage your calendar, triage your email, run scripts, and even browse the web. By late January 2026, it had exploded. It gained 25,000 GitHub stars in a single day and surpassed React's star count within two months, a milestone that took React over a decade. By mid-February, Steinberger had joined OpenAI, and the project moved to an open source foundation under its final name: OpenClaw .

What was so special about OpenClaw? Why did this one take off when so many agent projects before it didn't?

Autonomous AI isn't new

Where we are today feels similar to April 2023 when AutoGPT hit the scene. It had the same GitHub trajectory with its promise of autonomous AI. Then reality hit. Agents got stuck in loops, hallucinated a lot, and racked up token costs. It didn't take long for people to walk away.

OpenClaw has one critical advantage: The models have gotten better. Recent LLMs like Claude Opus 4.6 and GPT-5.4 allow models to chain tools together, recover from errors, and plan multistep strategies. Steinberger's weekend project benefited from timing as much as design.

The architecture is intentionally simple. There are no vector databases and no multi-agent orchestration frameworks. Persistent memory is Markdown files on disk. Let me repeat that: Persistent memory is Markdown files on disk! The agent can read yesterday's notes and search its own files for additional context. You can view and edit the agent's files as needed. There's a useful lesson in that: Not every agent system needs a complex memory strategy. It's more important that you understand what the agent is doing and that it retains context across runs.

What fascinates me about OpenClaw is that none of the individual pieces are new. Persistent memory across sessions? We've been building that for years. Cron jobs to trigger agent actions on a schedule? Decades old infrastructure. Plug-in systems for extensibility? A very standard pattern. Webhooks into WhatsApp and Telegram? There are well-documented APIs for that. What Steinberger did was wire them together at the exact moment the underlying models could execute on multistep plans. The combination created something that felt quite different from anything that had come before.

Why this time feels different

OpenClaw nailed three things that previous agent projects missed: proximity, creativity, and extensibility.

Proximity—it lives where you already are every day. OpenClaw connects to WhatsApp, Slack, Discord, Telegram, and Signal. That single design decision changed its trajectory. The agent becomes an active participant in your workflow. People use it to manage their sales pipeline, automate emails, and kick off code reviews from their phones.

Next, it's proactive. OpenClaw doesn't wait for you to ask; it uses cron jobs to run tasks on a set schedule. It can check your email every day at 6am, draft a reply before you wake up, and even send it for you. And it reaches out when anything needs your attention. Agents become part of everyday life when integrated into familiar channels.

> *In no time, I had my agent, Jarvis, writing code and submitting PRs, running my daily standups, and deploying my code to AWS using AWS CloudFormation and the AWS CLI.*
>
> Kesha Williams

And finally, my favorite, it's open and extensible. OpenClaw's plug-in system, called "skills," lets the community build and share modular extensions on ClawHub . There are thousands of skills ready to be plugged into your agent. Agents can even write their own new skills and use them going forward. That extensibility meant more skills, more users, and more attack surfaces, which we'll get to.

The community ran with it. A social network exclusively for AI Agents, Moltbook , launched in late January and grew to over 1.5 million agent accounts. One agent created a dating profile for its owner on MoltMatch and started screening matches without being asked.

I'll admit, I got swept up in it, but that's not surprising; I've always been an early adopter of emerging technology. I bought a Mac mini, installed OpenClaw, and connected it to my Jira, AWS, and GitHub accounts. In no time, I had my agent, Jarvis, writing code and submitting PRs, running my daily standups, and deploying my code to AWS using AWS CloudFormation and the AWS CLI.

I spent a lot of time binding the gateway to localhost, auditing every skill, and restricting filesystem permissions. For me, hardening the setup was not optional. I'm now deploying via AWS Lightsail, which adds network isolation and managed security layers that are hard to replicate on a Mac mini in your home office.

The security problem no one wants to talk about

OpenClaw requires root-level access to your system by design. It needs your email credentials, API keys, calendar tokens, browser cookies, filesystem access, and terminal permissions. If you're like me, that would keep you up at night.

Security researchers found 135,000 OpenClaw instances exposed on the open internet, over 15,000 vulnerable to remote code execution. The default configuration binds

Bogus Avast website fakes virus scan, installs Venom Stealer instead

Malwarebytes Labs

A fake website impersonating Avast antivirus is tricking people into infecting their own computers.

The site looks legitimate, runs what appears to be a virus scan, and claims your system is full of threats. But the results are fake: when you're prompted to "fix" the problem, the download you're given is actually Venom Stealer —a type of malware designed to steal passwords, session cookies, and cryptocurrency wallet data.

This is a classic scare-and-fix scam: create panic, then offer a solution. In this case, the "solution" abuses the trusted Avast brand to deliver the attack.

A scan that finds exactly what the attacker wants you to see

The phishing page is a recreation of the Avast brand, complete with navigation bar, logo, and reassuring certification badges. Visitors are invited to run what appears to be a comprehensive virus scan. Once they click, the page stages a brief animation before delivering its predetermined verdict: three threats found, three threats removed, system protected. A scrolling console log names a specific detection— Trojan: Win32/Zbot. AA!dll —to lend the performance an air of specificity. The victim is then prompted to download the cure: a file called Avast_system_cleaner.exe .

This is the payload. And far from cleaning anything, it immediately begins stealing.

A Chrome service that is not Chrome

When the victim launches Avast_system_cleaner.exe , the binary—a 64-bit Windows PE executable roughly 2 MB in size—copies itself into a location designed to blend in with legitimate software: C:\Program Files\Google\Chrome\Application\v20svc.exe . The dropped file is byte-for-byte identical to the parent, sharing the same MD5 hash (0a32d6abea15f3bfe2a74763ba6c4ef5). It then launches the copy with the command-line flag –v20c , a meaningless argument whose sole purpose is to signal to the malware that it is running in its second-stage role.

The disguise is deliberate. A process named v20svc.exe sitting inside Chrome's application directory looks, at a glance, like a legitimate browser service component. Anyone scanning their task manager would likely scroll past it without a second thought. This is a textbook example of masquerading: naming a malicious binary to match the conventions of trusted software so it escapes casual inspection.

A debug artifact baked into the binary confirms its lineage: the PDB path reads crypter_stub.pdb , indicating the executable was packed using a crypter, which is a tool designed to scramble a payload's code so antivirus engines cannot recognise it from its signature alone. At the time of analysis, only 27% of engines on VirusTotal flagged the sample, meaning roughly three in four commercial antivirus products missed it entirely.

YARA rules matched the sample to the Venom Stealer malware family, a known descendant of the Quasar RAT framework that has been sold on underground forums since at least 2020. Venom Stealer is purpose-built for data theft: browser credentials, session cookies, cryptocurrency wallets, and credit card details stored in browsers.

Every cookie, every wallet, every saved password

Once running, the malware works through a checklist of high-value targets on the victim's machine.

It starts with browsers. Behavioral analysis confirms the malware harvests saved credentials and session cookies. In the analysis environment, it was observed directly accessing Firefox's cookie database at C:\Users\ \AppData\Roaming\Mozilla\Firefox\Profiles\ \cookies.sqlite-shm . Process memory also contained fully-formed JSON structures with stolen cookie data from Microsoft Edge and Google Chrome, including active sessions for Netflix, YouTube, Reddit, Facebook, LinkedIn, AliExpress, Outlook, Adobe, and Google. Stolen session cookies give the attacker the ability to hijack authenticated browser sessions without needing the victim's password, including sessions protected by two-factor authentication.

The malware also targets cryptocurrency wallets. Behavioral signatures confirm it searches for and attempts to steal locally-stored wallet data, and Venom Stealer is documented as targeting desktop wallet applications. For anyone holding crypto assets on a hot wallet, the implications are immediate.

Beyond credentials, the stealer captures a screenshot of the victim's desktop, saved temporarily as C:\Users\ \AppData\Local\Temp\screenshot_5sIczFxY95t2IQ5u.jpg , and writes a session tracking file to C:\Users\ \AppData\Roaming\Microsoft\fd1cd7a3\sess . A small marker file is also dropped at C:\Users\Public\NTUSER.dat —a path chosen to mimic a legitimate Windows registry hive file and avoid suspicion.

Disguised as analytics, delivered over plain HTTP

All stolen data is exfiltrated to a single command-and-control domain: app-metrics-cdn[.]com , which resolved to 104.21.14.89 (a Cloudflare address) during analysis. The domain name is crafted to look like a benign analytics or content delivery service, the kind of traffic that might not raise alarm bells in a corporate proxy log.

The exfiltration follows a structured four-step sequence over unencrypted HTTP. First, a multipart form-data POST to /api/upload transmits the collected file—screenshots, wallet data, cookie databases—totalling around 140 KB. A second POST to / api/upload-json sends a structured JSON payload of approximately 29 KB containing parsed credentials and cookies. A confirmation POST to /api/upload-com-

Oracle's Java 11 trap - Use OpenJDK instead!

Stephen Colebourne · Stephen Colebourne (Joda)

TL: DR; Java is still available at zero-cost , you just need to stop using Oracle JDK and start using an OpenJDK build, such as this one or this one .

The trap

Java 11 has been released . It is a major release because it has long-term support (LTS). But Oracle have also set it up to be a trap (either deliberately or accidentally).

For 23 years, developers have downloaded the JDK from Oracle and used it for $free. Type "JDK" into your favourite search engine, and the top link will be to an Oracle Java SE download page (I'm deliberately not providing a link). But that search and that link is now a trap.

Oracle JDK, the one all web searches take you to, is now commercial not $free.

The key part of the terms is as follows:

You may not: use the Programs for any data processing or any commercial, production, or internal business purposes other than developing, testing, prototyping, and demonstrating your Application;

The trap is as follows:

Download Oracle JDK (because that is what you've always done, and it is what the web-search tells you)

Use it in production (because you didn't realise the license changed)

Get a nasty phone call from Oracle's license enforcement teams demanding lots of money

In other words, Oracle can rely on inertia from Java developers to cause them to download the wrong (commercial) release of Java. Unless you read the text/warnings/legalese very carefully you might not even realise Oracle JDK is now commercial, and that you are therefore liable to pay Oracle for using this particular JDK in production.

(Update, 2018-10-03: Searches for Java 11 and JDK 11 now seem to be resolving to OpenJDK builds, not commercial ones!)

Is this trap malicious behaviour on the part of Oracle? Readers will have their own opinions. I do suggest bearing in mind that Oracle invests huge amounts in developing Java, so it is reasonable to have a commercial plan available for those that want it. And they do provide a $free alternative completely valid for commercial use...

The solution

The solution is simple!

Use an OpenJDK build.

There are many different $free OpenJDK builds of Java 11, so you need to choose the one that best fits your needs.

The Adoptium (formerly AdoptOpenJDK) build is $free, GPL licensed (with Classpath exception so safe for commercial use), and a good choice as it is vendor-neutral and is intended to have 4+ years of security patches.

Download $free Java from Adoptium here

The OpenJDK build from Oracle is $free, GPL licensed (with Classpath exception so safe for commercial use), and provided alongside their commercial offering. It will only have 6 months of security patches, after that Oracle intends you to upgrade to Java 12 .

Download $free Java from Oracle here

Many more OpenJDK builds are available, including ones available via your package manager. See this post for a list covering the wide variety of OpenJDK builds . And see my post on zero-cost Java for background info.

Download other OpenJDK builds here

And for a counterpoint, see Marcus' great summary of why the underlying changes here are actually good news.

Do NOT download or use the Oracle JDK unless you intend to pay for it.

For Java 11, download and use an OpenJDK build, from AdoptOpenJDK , Oracle or elsewhere.

(No comments on this post. There are plenty of other places to express opinions.)

Loading Smarter: SVG vs. Raster Loaders in Modern Web Design

Mariana Beldi · CSS-Tricks

I got this interesting question in an SVG workshop: "What is the performance difference between an SVG loader and simply rotating an image for a loader?"

The choice between Scalable Vector Graphics (SVG) and raster image loaders involves many factors like performance, aesthetics, and user experience. The short answer to that question is: there's almost no difference at all if you are working on something very small and specific. But let's get more nuanced in this article and discuss the capabilities of both formats so that you can make informed decisions in your own work.

Understanding the formats

SVGs are vector -based graphics, popular for their scalability and crispness. But let's start by defining what raster images and vector graphics actually are.

Raster images are based on physical pixels. They contain explicit color information for every single pixel. What happens is that you send the entire pixel-by-pixel information, and the browser paints each pixel one by one, making the network work harder.

This means:

they have a fixed resolution (scaling can introduce blurriness),

the browser must decode and paint each frame , and

animation usually means frame-by-frame playback , like GIFs or video loops.

Vectors are mathematical instructions that tell the computer how to draw a graphic. As Chris Coyier said in this CSS Conf : "Why send pixels when you can send math?" So, instead of sending the pixels with all the information, SVG sends instructions for how to draw the thing. In other words, let the browser do more and the network do less.

Because of this, SVGs:

scale infinitely without losing quality,

can be styled and manipulated with CSS and JavaScript, and

can live directly in the DOM, eliminating that extra HTTP request.

The power of vectors: Why SVG wins

There are several reasons why it's generally a good idea to go with SVG over raster images.

1. Transparency and visual quality

Most modern image formats support transparency, but not all transparency is equal. GIFs, for example, only support binary transparency , which means pixels are either fully transparent or fully opaque.

This often results in jagged edges at larger scales, especially around curves or on opaque or transparent backgrounds. SVGs support true transparency and smooth edges, which makes a noticeable difference for loaders that sit on top of complex UI layers.

JPG GIF PNG SVG Vector ☒ ☒ ☒ ☒ Raster ☒ ☒ ☒ ☒ Transparency ☒ ☒ ☒ ☒ Animation ☒ ☒ ☒ ☒ Compression Lossy Lossless Lossless Lossless

2. "Zero request" performance

From a raw performance perspective, rotating a small PNG and an SVG in CSS (or JavaScript for that matter) is similar. SVGs, however, win in practice because they are gzip-friendly and can be embedded inline .

By pasting the SVG code directly into your HTML, you eliminate an entire HTTP request. For something like a loader — a thing that's supposed to show up while other things are loading — the fact that SVG code is already there and renders instantly is a huge win for performance.

More importantly, loaders affect perceived performance . A loader that adapts smoothly to its context and scales correctly can make wait times feel shorter, even if the actual load time is the same.

And even though the SVG code looks like it would be heavier than a single line of HTML, it's the image's file size that truly matters. And the fact that we're measuring SVG in bytes that can be gzipped means it's a lot less overhead in the end.

All that being said, it is still possible to import an SVG in an just like a raster file (among a few other ways as well):

And, yes, that does count as a network request even though it respects the vector-ness of the file when it comes to crisp edges at any scale. That, and it eliminates other benefits, like the very next one.

3. Animation, control, and interactivity

Loaders formatted in SVG are DOM-based, not frame-based. That means you can:

change colors via currentColor (like the example above),

react to application state or user interaction,

respect user prefers, like prefers-reduced-motion and prefers-color-scheme , and

modify shapes directly in code without needing to export new assets

[illegible]

The March Madness scam playbook

Malwarebytes Labs

March Madness is the annual men's and women's NCAA Division I basketball tournament, where 68 teams play in a single-elimination bracket for the US national championship.

But March Madness doesn't just bring buzzer beaters and busted brackets. It also kicks off a short, intense season for scammers who know fans are distracted, emotional, and often in a hurry. In this post, we'll walk through the main scam patterns that pop up around the NCAA men's basketball tournament, so you can recognize them and shut them down before they score.

Large sporting events combine three components that scammers love: money, emotion, and urgency. Fans are hunting for last-minute tickets, "can't-miss" bets, and ways to watch every game. They're in a hurry, so their guard is down.

From an attacker's perspective, March Madness is conveniently predictable. Every year in March, millions of people will Google the same terms, click the same types of ads, and respond to the same social media bait. Once you've seen the patterns, you'll start recognizing them around other major events too.

Fake ticket marketplaces and resale fraud

Ticket scams are a staple of any big concert, playoff series, or tournament, and March Madness is no exception.

The playbook is simple: Scammers set up sites and listings that look like legitimate ticket resellers, then take your money and run.

Things to keep an eye on:

Screenshots or PDFs of barcodes won't work when entry tickets are dynamic or tied to an app, but scammers still sell them. Victims will only find out at the gate when tickets are rejected.

Too good to be true last-minute deals. Offers for prime seats at prices well below the official box office, often paired with urgency imposing tactics: "must pay in the next 10 minutes," "three buyers waiting," or "I'll lose my deposit if you don't decide now."

Sellers push victims into private channels (text, WhatsApp, DMs) and insist on payment methods that are irreversible, like wire transfers, P2P apps, gift cards, or cryptocurrencies.

Fake betting sites, "sure thing" tips, and bonus traps

Legal sports betting has gone mainstream in the US, and March Madness is one of its biggest events. The huge number of casual bettors is a scammer's dream. Their tactics can be divided into two main categories:

Cloned betting platforms. Attackers create sites and apps that mimic real sportsbooks, complete with copied logos, odds feeds, and login pages. Users deposit funds, place bets, and maybe even see "winnings" pile up in the interface—until they try to withdraw and are hit with fees, extra deposits, silent account bans, or witness a disappearing act.

"Guaranteed" bets. Social media fills up with self-proclaimed experts selling access to VIP betting groups or "guaranteed" locks on tournament games. Victims pay for tips or are funneled into shady offshore sites that conveniently lose their money or demand more deposits to "unlock" withdrawals.

Streaming scams

Not everyone has a cable subscription or an official streaming package, and scammers know many fans will look for free or cheap alternatives. That creates a fertile ground for malicious streaming offers. There are some common patterns to watch for:

Fake portals promising all the games live. Websites advertise free HD streams of every tournament game but require you to create an account and enter a credit card "for age verification" or a "free trial." Once you submit details, charges appear or your card data is sold on.

Malicious players and extensions. Some sites will prompt you to download a special player, codec, or browser extension before you can watch. Instead of video, you get adware, browser hijackers, or a foothold for more serious malware.

Shortened URLs and reposted "official stream" links spread quickly around tip-off time, often redirecting through multiple ad and tracking networks before landing on phishing or scam pages.

Like-farming and other social media clickbait promising free streams only to boost the account's reputation for a next wave of scams.

Bracket phishing, office pools, and prize scams

Brackets are part of the culture: friends, families, and workplaces run pools where everyone predicts the tournament results. Scammers piggyback on that habit with phishing campaigns and fake prize draws. These usually show up in a few ways:

Official bracket challenge phishing. Emails or messages invite you to join a tournament bracket hosted by a big brand or media outlet, complete with logos and plausible wording. The link really leads to a credential-harvesting page masquerading as your email, work SSO, or a well-known sports site.

Fake "you won the pool" notifications. Messages claim

Product-market fit methodology for early-stage devtool companies

Irina Nazarova · Evil Martians Chronicles

Author: Irina Nazarova, CEO

Topics: Developer Community, Devtools startup advisory

How do you measure product-market fit for a developer tool? A PMF scoring model from Evil Martians—a product development consultancy for developer tools startups—built on data from 37 devtools companies across AI, infrastructure, and cybersecurity. Five metrics, real benchmarks, and a dual score that tells you whether to invest in product or go-to-market.

Everyone talks about product-market fit. But what does it actually mean for a developer tools startup? “Are we close?” “Getting closer?” “What should we focus on to get closer?” At Evil Martians, we’ve spent nearly 20 years working with devtools companies—from pre-seed through Series B and beyond—and we kept hearing these same questions. So we built a scoring model that actually answers them, grounded in data from 37 real companies, and put it right on our homepage.

Read more

Pairing Guidelines

Robert C Martin (Clean Coder)

Everybody pairs from time to time. It is a rare programmer who has not sat down with another programmer to look something over or help find a bug.

Deep problems, that require much heavy thinking, do not often lend themselves to pairing. The interaction between the programmers tends to disrupt the necessary concentration.

On the other hand, it is not uncommon for programmers to get caught in a problem that they think is deep, but for which there is a much simpler solution that another programmer could quickly see. So it is wise to start deep problems with a pair, or even a mob, but then break it up when it becomes clear that the problem is irreducible.

On the other side of the spectrum, there is no good reason to pair on trivial matters. Fleshing out a list of error messages, or loading fifty fields into a form are relatively mindless activities that do not require the scrutiny afforded by pairing.

Then there is the vast middle. This is where pairing/mobbing are most valuable. These are problems that are non-trivial, but also not particularly deep. This is 90% of all programming. Pairing on this type of code keeps that code well tested, well structured, and as simple as possible.

Pairing should always be voluntary, never be forced, never be scheduled by a manager, and never tracked. It is an informal process that is entirely under the control of the individual programmers.

Some people can’t, or won’t do it. That’s ok; but it may require that their participation in certain projects be curtailed.

Pairing sessions should be short-ish. 20-40 minutes at a time. (Tomato sized) With no more than three or four consecutive sessions of that length. This is not a rule, just an informal guideline.

Not all code that would benefit from pairing, should be written by pairs. A mature team might pair 50% of the time, or even less. During the pairing sessions, a large amount of code will be reviewed; far more than the pair is actively writing; and thus the benefits of pairing will be seen in very large swathes of non-paired code.

Bottom line: Don’t be a jerk. Pair sometimes, don’t pair other times. Pair enough so that you have a good grasp of the overall system, and know enough of what your teammates are doing that you could step into their roles if the need arose. Don’t pair so much that you hate your job, and your teammates.

ANALYSIS

What's !important #6: :heading, border-shape, Truncating Text From the Middle, and More

Daniel Schwarz · CSS-Tricks

Despite what's been a sleepy couple of weeks for new Web Platform Features, we have an issue of What's !important that's prrrretty jam-packed . The web community had a lot to say, it seems, so fasten your seatbelts!

@keyframes animations can be strings

Peter Kröner shared an interesting fact about @keyframes animations — that they can be strings:

@keyframes "@animation" { /* … */ }

#animate-this { animation: "@animation"; }

Yo dawg, time for a #CSS fun fact: keyframe names can be strings. Why? Well, in case you want your keyframes to be named "@keyframes," obviously!

#webdev

[image or embed]

— Peter Kröner (@sirpepe.bsky.social) Feb 18, 2026 at 10:33

I don't know why you'd want to do that, but it's certainly an interesting thing to learn about @keyframes after 11 years of cross-browser support!

: vs. = in style queries

Another hidden trick, this one from Temani Afif , has revealed that we can replace the colon in a style query with an equals symbol . Temani does a great job at explaining the difference, but here's a quick code snippet to sum it up:

. Jay-Z { –Problems: calc(98 + 1);

/* Evaluates as calc(98 + 1), color is blueivy */ color: if(style(–Problems: 99): red; else: blueivy);

/* Evaluates as 99, color is red */ color: if(style(–Problems = 99): red; else: blueivy); }

In short, = evaluates –Problems differently to : , even though Jay-Z undoubtably has 99 of them (he said so himself).

Declarative s (and an updated .visually-hidden)

David Bushell demonstrated how to create s declaratively using invoker commands , a useful feature that allows us to skip some J'Script in favor of HTML, and works in all web browsers as of recently.

Also, thanks to an inquisitive question from Ana Tudor, the article spawned a spin-off about the minimum number of styles needed for a visually-hidden utility class . Is it still seven?

Maybe not…

How to truncate text from the middle

Wes Bos shared a clever trick for truncating text from the middle using only CSS:

Someone on reddit posted a demo where CSS truncates text from the middle.

They didn't post the code, so here is my shot at it with Flexbox

[image or embed]

— Wes Bos (@wesbos.com) Feb 9, 2026 at 17:31

Donnie D'Amato attempted a more-native solution using ::highlight() , but ::highlight() has some limitations, unfortunately. As Henry Wilkinson mentioned , Hazel Bachrach's 2019 call for a native solution is still an open ticket, so fingers crossed!

How to manage color variables with relative color syntax

Theo Soti demonstrated how to manage color variables with relative color syntax . While not a new feature or concept, it's frankly the best and most comprehensive walkthrough I've ever read that addresses these complexities.

How to customize lists (the modern way)

In a similar article for Piccalilli, Richard Rutter comprehensively showed us how to customize lists , although this one has some nuggets of what I can only assume is modern CSS. What's symbols() ? What's @counter-style and extends ? Richard walks you through everything .

Source: Piccalilli .

Can't get enough on counters? Juan Diego put together a comprehensive guide right here on CSS-Tricks .

How to create typescales using :heading

Safari Technology Preview 237 recently began trialing :heading / :heading() , as Stuart Robson explains . The follow-up is even better though, as it shows us how pow() can be used to write cleaner typescale logic, although I ultimately settled on the old-school – elements with a simpler implementation of :heading and no sibling-index() :

:root { –font-size-base: 16px; –font-size-scale: 1.5; }

:heading { /* Other heading styles */ }

/* Assuming only base/h3/h2/h1 */

body { font-size: var(–font-size-base); }

h3 { font-size: calc(var(–font-size-base) * var(–font-size-

Leveraging the Plain Old Python Function

Stitch Fix Multithreaded

The role of the full-stack-data-scientist is not what it once was.

With the advent of more powerful tooling, new industry standards in MLOps, and greater investment in platforms, the day-to-day of a data scientist has changed significantly at Stitch Fix. The difference, however, is subtle. The structure of their job remains the same – engineers still do not write ETLs and data scientists function as generalists , but they now have to think on a higher level. Their job is constantly getting more and more complex—the business needs are in flux and the infrastructure they use is more powerful than it ever was. The old strategy of cobbling together complex systems will only end in stressed-out data scientists with too much infrastructure on their plate.

To avoid this cycle of complexity, Stitch Fix invests in a platform team to innovate new ways of supporting a data scientist's engineering needs. Rather than constructing custom model-deployment mechanisms, building microservices from the ground up, and managing highly interdependent chains of data transformations, data scientists at Stitch Fix can leverage powerful infrastructure by constructing plain old Python functions to represent their needs .

In this blog post we're going to take a different approach than usual. Rather than digging into a specific piece of technology, we'll present our philosophy of functions for data science APIs and back it up with some motivating examples. We'll explain the power of functions as a DSL, share some successes we've had using functional interfaces to build our MLOps stack, and connect our approach with external, open-source frameworks that the industry is beginning to adopt. Our goal is to convince you that a function-first approach will enable data practitioners to do more while doing less. The functional approach allows them to plug into the business in a scalable manner while avoiding the complexity of managing infrastructure and architectural decisions.

On Functions and Functionality

Since way before many of us were born, the software industry has been battling between functional and procedural programming paradigms. Do you offer imperative commands over the operations of your system? Or do you declare the intent and let some system in the background handle the operations for you? While we won't opine on this age-old debate, we do bring this up for a reason. At Stitch Fix, data scientists get the best of both worlds:

They can declare infrastructure and plug into the business by writing python functions to fit into platform-provided frameworks

They can specify custom model and business logic by implementing said functions

At the core, we believe data scientists should only have to think about business logic, and all else (infrastructure, etc...) should be given to them on a silver platter. The delineation of responsibilities here is critical in providing high-power tooling to allow data scientists to plug into the business, and Python functions happen to be the perfect way to codify it. The name, type annotations, decorators, and parameters of a function all provide plenty of information to declare the structure of the systems they need. The guts of a function allow a data scientist to iterate on and cleanly represent their business logic.

@ do_something_fancy_with_function def my_function (param_1 : pd . Series , param_2 : int) -> pd . Series : “”“Does a thing with some params”“” return fancy_business_logic (param_1 , param_2)

Function Name

my_function

Type Annotations

pd. Series, int

Decorators

@do_something_fancy_with_function

Parameters

param_1, param_2

Docstring

“”“Does a thing with some params”“”

Logic

return fancy_business_logic(param_1, param_2)

How do we actually link this metadata to give data scientists MLOps capabilities? Let's go through a few examples...

Years ago, releasing a model at Stitch Fix took considerable effort. To execute a model in a production context (batch/online), data scientists had to:

Save the model blob to a blob store

Build a service or a batch job that exposed their model, with Python requirements that exactly matched training

Download the model into the appropriate production context, updating as needed

Maintain and manage the service/batch job

This regularly repeated chain of micromanaged infrastructure was not nimble enough to support the velocity at which data scientists at Stitch Fix work. So, we built new tooling that enables data scientists to publish their model to production by:

Saving a function using our API

More On Types

Robert C Martin (Clean Coder)

Recently I wrote a cute little program for doing Turtle Graphics . For those of you who don't know, turtle graphics were originally added to the LOGO language by Seymour Papert in the late 1960s. He built a robot that he called a "turtle" that could hold a pen. The robot had wheels and could move forwards and backwards, and could rotate left and right. It could also raise and lower the pen. When placed on a sheet of paper, the turtle could be commanded to draw interesting designs.

Papert's goal was to teach children about programming. As the years went by the robot got replaced with screens, and the turtle became an icon that could draw lines. Children from the 70s until now have been enthralled by the simple commands for directing the turtle, and the elegant drawings they can make.

For example, this is how you might draw a square:

forward 100 right 90 forward 100 right 90 forward 100 right 90 forward 100 right 90.

Recently I had a need to explore some interesting geometrical designs. Turtle graphics would be perfect for my purposes. So I wrote a turtle graphics processor in Clojure. [code]

I used the quil framework which is based on the Processing framework in Java. This framework makes it very easy to create simple GUIs in Clojure.

Now consider the problem of the Turtle. What is the type model for this object? What fields does it have, and what constraints must be placed on those fields?

Here was my solution to that problem, written in clojure/spec . As usual, in Clojure, you start at the bottom and read towards the top.

(s/def ::position (s/tuple number? number?)) (s/def ::heading (s/and number? #(

(defn make [] {:post [(s/assert ::turtle %)]} {:position [0.0 0.0] :heading 0.0 :velocity 0.0 :distance 0.0 :omega 0.0 :angle 0.0 :pen :up :weight 1 :speed 5 :visible true :lines [] :state :idle})

This is the default constructor of the turtle. Notice that it just loads up all the required fields into a map. Notice also that there is a post condition that asserts that the result conforms the the turtle type.

This is nice. If I forget to initialize a field, or if I initialize a field to a value that conflicts with the type, I get an error.

Here's another, more complex example. Don't freak out, you don't have to understand this in detail.

(defn update-turtle [turtle] {:post [(s/assert ::turtle %)]} (if (= :idle (:state turtle)) turtle (let [{:keys [distance state angle lines position pen pen-start] :as turtle} (-> turtle (update-position) (update-heading)) done? (and (zero? distance) (zero? angle)) state (if done? :idle state) lines (if (and done? (= pen :down)) (conj lines (make-line turtle)) lines) pen-start (if (and done? (= pen :down)) position pen-start)] (assoc turtle :state state :lines lines :pen-start pen-start))))

This is the function that updates the turtle for each screen refresh. Again, notice the post condition . If anything is calculated incorrectly by the update-turtle function, I'll get an exception right away.

Now some of you might be worried that by checking types at runtime I could end up with runtime errors in production. You might therefore assert that static typing is better because the compiler checks the types long before the program ever executes.

However, I do not intend to have runtime errors in production, because I have a suite of tests that exercise all the behaviors of the system. Here's just one of those tests:

(describe "Turtle Update" (with turtle (-> (t/make) (t/position [1.0 1.0]) (t/heading 1.0))) (context "position update" (it "holds position when there's no velocity" (let [turtle (-> @turtle (t/velocity 0.0) (t/state :idle)) new-turtle (t/update-turtle turtle)] (should= turtle new-turtle)))

Again, you don't have to understand this in any detail. Just notice that the make and update-turtle functions are being invoked. Since those functions have post conditions that will check the types, and since my suite of tests is exhaustive, I am quite certain that there will be no runtime errors in production and that my dynamic type checking is as robust as any static type system.

The dynamic nature of the type checking allows me to assert type constraints that are very difficult, if not impossible, to assert at compile time. I can, for example, assert complex relationships between the values of the fields.

To expand on that example, imagine the type model of an accounting balance sheet. The sum of the assets, liabilities and equities on the balance sheet must be zero. This is easy to assert in clojure/spec but is difficult, if not impossible, to assert in most statically typed languages.

Moreover, I get to control when types are asserted. It is up to me to decide if and when a certain type should be checked. This gives me a lot of power and flexibility. It allows me to violate the type rules in the midst of computations, so long as the end result ends up conforming to the types.

One last point. In the late 90s and the 2000s, there was a lengthy and animated (and sometimes acrimonious) debate over TDD vs DBC (Design by Contract). What clojure/spec has taught me is that the two play very well together, and both should be in every programmer's toolkit.

REPL Driven Design

Robert C Martin (Clean Coder)

If you follow me on facebook you know that I've been publishing daily CoronaVirus statistics. I generate these statistics using the daily updates in the Johns Hopkins github repository .

At first I just hand copied the data into a spreadsheet. But that became tedious quite rapidly.

Then, in late March, I wrote a little Clojure program to extract and process the data. Every morning I pull the repo, and then run my little program. It reads the files, does the math, and prints the results.

Of course I used TDD to write this little program.

But over the last several weeks I've made quite a few small modifications to the program; and it has grown substantially. In making these adaptations I chose to use a different discipline: REPL Driven Design.

REPL Driven Design is quite popular in Clojure circles. It's also quite seductive. The idea is that you try some experiments in the REPL to make sure you've got the right ideas. Then you write a function in your code using those idea. Finally, you test that function by invoking it at the REPL.

It turns out that this is a very satisfying way to work. The cycle time – the time between a code experiment and the test at the REPL – is nearly as small as TDD. This breeds lots of confidence in the solution. It also seems to save the time needed to mock, and create fake data because, at least in my case, I could use real production data in my REPL tests. So, overall, it felt like I was moving faster than I would have with TDD.

But then, in late April, I wanted to do something a little more complicated than usual. It required a design change to my basic structure. And suddenly I found myself full of fear. I had no way to ensure that those design changes wouldn't leave the system broken in some way. If I made those changes, I'd have to examine every output to make sure that none of them had broken. So I postponed the change until I could muster the courage, and set aside the dedicated time it would require.

The change was not too painful. Clojure is an easy language to work with. But the verfication was not trivial, which led me to deploy the program with a small bug – a bug I caught 4 days later. That bug forced me to go back and correct the data and graphs that I generated.

Why did I need the design change? Because I was not mocking and creating fake data. My functions just read from the repo files directly. There was no way to pass them fake data. The design change I needed to make was precisely the same as the design change that I'd have needed for mocking and fake data.

Had I stuck with the TDD discipline I would have automatically made that design change, and I would not have faced the fear, the delay, and the error.

Is it ironic that the very design change that TDD would have forced upon me was the design change I eventually needed? Not at all. The decoupling that TDD forces upon us in order to pass isolated inputs and gather isolated outputs is almost always the design that fascilitates flexibility and promotes change.

So I've learned my lesson. REPL driven development feels easier and faster than TDD; but it is not. Next time, it's back to TDD for me.

How to Favicon in 2026: Three files that fit most needs

Andrey Sitnik · Evil Martians Chronicles

Author: Andrey Sitnik, Author of PostCSS and Autoprefixer, Principal Frontend Engineer

Topic: CSS

Prefer SVG over PNG, trust browsers to downscale, drop obscure formats—the ultimate, exhaustive guide to favicons for modern web. Includes steps for static HTML and Webpack.

It's time to rethink how we cook a set of favicons for modern browsers and stop the icon generator madness. Frontend developers currently have to deal with 20+ static PNG files just to display a tiny website logo in a browser tab or on a touchscreen. Read on to see how to take a smarter approach and adopt a minimal set of icons that fits most modern needs.

Read more

Life's too short to hand-write API types: OpenAPI-driven React

Travis Turner · Evil Martians Chronicles

Authors: Yuri Mikhin, Frontend Engineer, and Travis Turner, Tech Editor

Topics: React, TypeScript

Transform your React development with contract-first APIs. Generate TypeScript types, build type-safe clients, and develop with mocks while backend implements—no more waiting, no more integration chaos.

Most React apps have a problem: frontend types and backend reality drifting apart. You copy API shapes into TypeScript files, backend changes something, and you find out in production. This guide fixes that by making your OpenAPI spec the source of truth—generating types, clients, and validation schemas automatically so contract mismatches break builds instead of production. You'll also set up network-level mocks so your team can build and test features against realistic API behavior without waiting on backend to deploy anything.

Read more

Algo Hour - Large Scale Data & ML Monitoring with whylogs | Alessya Visnjic

Stitch Fix Multithreaded

Title:

Large Scale Data & ML Monitoring with whylogs

Talk Abstract:

In the era of microservices, decentralized ML architectures and complex data pipelines, data quality has become a bigger challenge than ever. When data is involved in complex business processes and decisions, bad data can, and will, affect the bottom line. As a result, ensuring data quality across the entire ML pipeline is both costly, and cumbersome while data monitoring is often fragmented and performed ad hoc. An open source library called whylogs is built to address these challenges. It is a lightweight data profiling library that enables end-to-end data monitoring across the entire software stack. The library implements a language and platform agnostic approach to data quality and data monitoring. It's been deployed at massive-scale data environments, on structured and unstructured data modalities, and across a range of points in the ML lifecycle. In this talk, we will provide an overview of the whylogs architecture, including its lightweight statistical data collection approach and we will show how users can apply this library to existing data and ML pipelines.

Date and Time:

The talk will be held on Tuesday, October 11th at 1:00PM PDT.

Recording Info:

This talk was recorded live and is viewable below:

Speaker Info:

Alessya Visnjic is the CEO of WhyLabs, the AI Observability company building tools that power robust and responsible AI deployment. Prior to WhyLabs, Alessya was a CTO-in-residence at the Allen Institute for AI, where she evaluated commercial potential for the latest AI research. Earlier, Alessya spent 9 years at Amazon leading ML initiatives, including forecasting and data science platforms. Alessya is also the founder of Rsqrd AI, a global community of 1,000+ AI practitioners who are committed to making enterprise AI technology responsible.

BRIEFS

Faouz EL FASSI, Drivy / Getaround Tech: In the first of this series of blog posts about Data-Warehousing, I've been talking about how we use and manage our Amazon Redshift cluster at Drivy.

Thibaud Esnouf, Drivy / Getaround Tech: As a JavaScript developer, you surely have encountered some functions that require a lot of arguments to be called. Because the argument list is an Array-like object, all the values need to be set and so it may have given you a headache to understand the order and purpose of each argument.

Nicolas Zermati, Drivy / Getaround Tech: A few months ago we faced a memory issue on some of our background jobs. Heroku was killing our dyno because it was exceeding its allowed memory. Thanks to our instrumentation of Sidekiq , it was easy to spot the culprit. The job was doing a fairly complex SQL request, and outputing the query's result into a CSV file before archiving this file.

The Hacker News, The Hacker News (cybersecurity): The North Korean threat actors behind the Contagious Interview campaign, also tracked as WaterPlum, have been attributed to a malware family tracked as StoatWaffle that's distributed via malicious Microsoft Visual Studio Code (VS Code) projects. The use of VS Code "tasks.json" to distribute malware is a relatively new tactic adopted by the threat actor since December 2025, with the attacks

Travis Turner, Evil Martians Chronicles: Authors: Victoria Melnikova, Head of New Business, Host of Dev Propulsion Labs, and Travis Turner, Tech Editor

Topics: Case Study, Developer Community

Evil Martians is a developer tools consultancy. Founded in 2006, we work with about 40 early-stage startups a year, mostly seed to Series B. We helped bolt.new scale from zero to $40M+ ARR in 5 months. Teleport and Tines reached unicorn status with Martians on their teams.

Established nearly 20 years ago, Evil Martians is a trusted software development consultancy for developer tools startups.

Read more

The Hacker News, The Hacker News (cybersecurity): Cybersecurity researchers have discovered a new payment skimmer that uses WebRTC data channels as a means to receive payloads and exfiltrate data, effectively bypassing security controls. "Instead of the usual HTTP requests or image beacons, this malware uses WebRTC data channels to load its payload and exfiltrate stolen payment data," Sansec said in a report published this week. The attack,

Marc G Gauthier, Drivy / Getaround Tech: We've always valued releasing quickly, as unreleased code is basically inventory . It slowly gathers dust and becomes outdated or costs time to be kept updated. Almost 3 years ago we published an article " Drivy, version 500! " on our main blog, so I feel now is the time to get more into the details of how we accomplish pushing a lot of new versions of the app to production.

Chaim Gartenberg, The Keyword (Google Blog): Get the origin story behind the stunning wallpapers you see on Google TV Streamer, Nest Hub and other Google TV devices.

Renaud Boulard, Drivy / Getaround Tech: Android Makers is the largest Android event in France, organized by the PAUYG and BeMyApp . This year the event was held in Le Beffroi de Montrouge - what a great place to enjoy a conference. There was much more space than the previous Android Makers event: we were really comfortable, with all we needed to be fully focussed on the conferences.

The Hacker News, The Hacker News (cybersecurity): Trivy, a popular open-source vulnerability scanner maintained by Aqua Security, was compromised a second time within the span of a month to deliver malware capable of stealing sensitive CI/CD secrets. The latest incident impacted GitHub Actions "aquasecurity/trivy-action" and "aquasecurity/setup-trivy," which are used to scan Docker container images for vulnerabilities and set up GitHub Actions

The Hacker News, The Hacker News (cybersecurity): TeamPCP, the threat actor behind the recent compromises of Trivy and KICS, has now compromised a popular Python package named litellm, pushing two malicious versions containing a credential harvester, a Kubernetes lateral movement toolkit, and a persistent backdoor. Multiple security vendors, including Endor Labs and JFrog, revealed that litellm versions 1.82.7 and 1.82.8 were published on March

Stephan Lensky, HubSpot Product Blog:

This post is the second in a series about empowering product, UX, and engineering teams with AI, especially in the context of writing code. Read the first post about scaling AI adoption here !

Kartik Vishwanath, HubSpot Product Blog:

On October 20, 2025, HubSpot experienced a significant service disruption affecting multiple product features due to a severe AWS outage in the us-east-1 region. While our infrastructure remained intact, the widespread nature of the cloud provider failure impacted both our services and critical third-party vendors we rely on. We've completed a thorough analysis of this incident and are implementing comprehensive improvements to strengthen our resilience against future cloud provider disruptions.

The Hacker News, The Hacker News (cybersecurity): Rising geopolitical tensions are reflected (or in some cases preceded) by cyber operations, while technology itself has become politicized. Let's admit it: we are in the middle of it. Introduction: One tech power to rule them all is a thing of the past The relative safety, peace and prosperity that much of the world has enjoyed since 1945 was not accidental. It emerged from the ashes

Howard Wilson, Drivy / Getaround Tech: As developers, we sometimes find ourselves faced with feature requests that will take weeks of work and touch many areas of the codebase. This comes with increased risk in terms of how we spend our time and whether things break when we come to release.

Jordan Jalabert & Emily Fiennes, Drivy / Getaround Tech: After working as a teacher and translator for several years, Emily embarked on a new phase in her career by learning a different kind of language: programming.

join us!

where the camera app will save the picture, and using external storage might be tempting.

The Global Digest

Vol. 1, No. 007 · 2026-03-30

Curated and composed automatically.